University of the Philippines - Baguio

Archie Junio's Reference Notebook

29/04/2026

Contents

1. Data Structures 1
1.1. Segment Tree 1
1.2. Union Find 1
1.3. 2D Sparse Table 1
2. Dynamic Programming 1
2.1. Lichao Tree 1
2.2. Divide And Conquer 2
3. Geometry 2
3.1. Angles And Rotations 2
3.2. Distances 2
3.3. Intersections 2
3.3.1. Circle Line Intersection 2
3.3.2. Circle Circle Intersection 2
3.3.3. Line Intersect 2
3.4. Closest Pair 2
3.5. Convex Hull 2
3.5.1. Convex Hull 2
3.5.2. Convex Hull 3D 2
3.6. Areas 3
3.6.1. Triangle Area 3
3.6.2. Polygon Area 3
3.7. Dot Cross 3
3.8. Line Sweep Closest Pair 3
4. Graphs 3
4.1. Max Flow 3
4.1.1. Dinic 3
4.1.2. Edmonds Karp 3
4.1.3. Bipartite Matching 3
4.2. Shortest Path 4
4.2.1. Dijkstra 4
4.2.2. Bellman Ford 4
4.3. Minimum Spanning Tree 4
4.3.1. Kruskal Min Spanning Tree 4
4.3.2. Prim Min Spanning Tree 4
4.4. Least Common Ancestor 4
4.4.1. Binary Lifting Lca 4
4.4.2. Euler Lca 5
4.4.3. Tarjan Lca 5
4.5. Minimum Mean Weight Cycle 5
4.6. Strongly Connected Components 5
4.7. Heavy Light Decomposition 5
4.8. Euler Path/Cycle 6
4.8.1. Directed Euler 6
4.8.2. Undirected Euler 6
4.9. Tree Isomorphism 7
4.10. Centroid Decomposition 7
5. Math I Algebra 7
6. Math II Combinatorics 7
6.0.1. Lucas Theorem 7
6.0.2. Catalan Numbers 8
6.0.3. Kth Permutation 8
6.0.4. Factoradics 8
7. Math III Number Theory 8
7.0.1. Gcd Subset Counter 8
7.0.2. Linear Diophantine 8
7.0.3. Chinese Remainder Theorem 8
8. Math IV Numerical Methods 8
8.0.1. Fast Square Testing 8
8.0.2. Simpson Integration 8
9. Strings 8
9.0.1. Longest Common Prefix 8
9.0.2. Hasher 9
9.0.3. Z Algorithm 9
9.0.4. Aho Corasick Trie 9
9.0.5. Knuth Morris Pratt 9
9.0.6. Trie 9
9.0.7. Suffix Array 9
9.0.8. Booth Min String Rotation 10
10. Other Algorithms 10
10.0.1. Longest Increasing Subsequence 10
10.0.2. Cycle Find 10
10.0.3. 2Sat 10
10.0.4. Dpll 11

1. Data Structures

1.1. Segment Tree.

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (2 * self.n)
        for i in range(self.n):
            self.tree[self.n + i] = data[i]
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[i << 1] + self.tree[i << 1 | 1]

    def update(self, p, value):
        p += self.n
        self.tree[p] = value
        while p > 1:
            self.tree[p >> 1] = self.tree[p] + self.tree[p ^ 1]
            p >>= 1

    def query(self, l, r):
        res = 0
        l += self.n
        r += self.n
        while l < r:
            if l & 1:
                res += self.tree[l]
                l += 1
            if r & 1:
                r -= 1
                res += self.tree[r]
            l >>= 1
            r >>= 1
        return res
```

1.2. Union Find.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)

        if root_i != root_j:
            if self.size[root_i] < self.size[root_j]:
                root_i, root_j = root_j, root_i
            self.parent[root_j] = root_i
            self.size[root_i] += self.size[root_j]
            return True
        return False

    def get_size(self, i):
        return self.size[self.find(i)]
```

1.3. 2D Sparse Table.

```
import math

class SparseTable2D:
    def __init__(self, A):
        self.A = A
        self.n = len(A)
        self.m = len(A[0])

        self.LOGN = int(math.log2(self.n)) + 1
        self.LOGM = int(math.log2(self.m)) + 1

        self.lg = [0] * (max(self.n, self.m) + 1)
        for i in range(2, len(self.lg)):
            self.lg[i] = self.lg[i // 2] + 1

        self.st = [[[[0] * self.m for _ in range(self.n)] for _ in range(self.LOGM)] for _ in range(self.LOGN)]

        self.build()

    def build(self):
        for i in range(self.n):
            for j in range(self.m):
                self.st[0][0][i][j] = self.A[i][j]
```

```
-------------------------------------------------------------------------
------- for bj in range(1, self.LOGM): ------------------------------
----------- for i in range(self.n): ------------------------------------
--------------- for j in range(self.m - (1 << bj) + 1): ---------------
------------------ self.st[0][bj][i][j] = max( ----------------------
---------------------- self.st[0][bj - 1][i][j], ---------------------
---------------------- self.st[0][bj - 1][i][j + (1 << (bj - 1))] ---
------------------ ) ------------------------------------------------
-------------------------------------------------------------------------
------- for bi in range(1, self.LOGN): ------------------------------
----------- for i in range(self.n - (1 << bi) + 1): --------------------
--------------- for j in range(self.m): ------------------------------
------------------ self.st[bi][0][i][j] = max( ----------------------
---------------------- self.st[bi - 1][0][i][j], ---------------------
---------------------- self.st[bi - 1][0][i + (1 << (bi - 1))][j] ---
------------------ ) ------------------------------------------------
-------------------------------------------------------------------------
------- for bi in range(1, self.LOGN): ------------------------------
----------- for bj in range(1, self.LOGM): ------------------------------
--------------- for i in range(self.n - (1 << bi) + 1): --------------
------------------ for j in range(self.m - (1 << bj) + 1): ----------
---------------------- self.st[bi][bj][i][j] = max( ------------------
--------------------------- self.st[bi - 1][bj - 1][i][j], -----------
--------------------------- self.st[bi - 1][bj - 1][i + (1 << (bi -
↪  1))][j],
--------------------------- self.st[bi - 1][bj - 1][i][j + (1 << (bj -
↪  1))],
--------------------------- self.st[bi - 1][bj - 1][i + (1 << (bi -
↪  1))][j + (1 << (bj - 1))]
---------------------- ) -----------------------------------------------
-------------------------------------------------------------------------
--- def query(self, x1, y1, x2, y2): ----------------------------------
------- kx = self.lg[x2 - x1 + 1] ----------------------------------
------- ky = self.lg[y2 - y1 + 1] ----------------------------------
-------------------------------------------------------------------------
------- return max( -------------------------------------------------
----------- self.st[kx][ky][x1][y1], -----------------------------------
----------- self.st[kx][ky][x2 - (1 << kx) + 1][y1], -------------------
----------- self.st[kx][ky][x1][y2 - (1 << ky) + 1], -------------------
----------- self.st[kx][ky][x2 - (1 << kx) + 1][y2 - (1 << ky) + 1] --
------- ) -----------------------------------------------------------
# Input Format: st = SparseTable2D(grid) ----------------------------
#               print(st.query(1, 1, 2, 2)) --------------------------
```

2. Dynamic Programming

2.1. Lichao Tree.

```
class Line: -------------------------------------------------------------
--- __slots__ = ['m', 'c'] -------------------------------------------
--- def __init__(self, m=0, c=float('-inf')): ------------------------
------- self.m = m -------------------------------------------------
------- self.c = c -------------------------------------------------
-------  ------------------------------------------------------------
--- def eval(self, x): -----------------------------------------------
------- return self.m * x + self.c ---------------------------------
-------------------------------------------------------------------------
class Node: -------------------------------------------------------------
--- __slots__ = ['line', 'left', 'right'] ----------------------------
--- def __init__(self): ----------------------------------------------
------- self.line = Line() -----------------------------------------
------- self.left = None -------------------------------------------
------- self.right = None ------------------------------------------
-------------------------------------------------------------------------
class LiChaoTree: -------------------------------------------------------
--- def __init__(self, min_x=-10**12, max_x=10**12): -----------------
------- self.root = Node() -----------------------------------------
------- self.min_x = min_x -----------------------------------------
------- self.max_x = max_x -----------------------------------------
-------------------------------------------------------------------------
--- def add_line(self, m, c): ----------------------------------------
------- self._insert(self.root, self.min_x, self.max_x, Line(m, c)) --
-------------------------------------------------------------------------
--- def _insert(self, node, l, r, new_line): -------------------------
------- mid = (l + r) // 2 -----------------------------------------
-------  ------------------------------------------------------------
------- left_better = new_line.eval(l) > node.line.eval(l) ---------
------- mid_better = new_line.eval(mid) > node.line.eval(mid) -------
-------  ------------------------------------------------------------
------- if mid_better: ---------------------------------------------
----------- node.line, new_line = new_line, node.line ----------------
------- if r - l <= 1: ---------------------------------------------
----------- return -----------------------------------------------------
-----------  ----------------------------------------------------------
------- if left_better != mid_better: ------------------------------
----------- if not node.left: ------------------------------------------
--------------- node.left = Node() -----------------------------------
----------- self._insert(node.left, l, mid, new_line) ----------------
------- else: ------------------------------------------------------
----------- if not node.right: -----------------------------------------
--------------- node.right = Node() ----------------------------------
----------- self._insert(node.right, mid, r, new_line) ---------------
-------------------------------------------------------------------------
--- def query(self, x): ----------------------------------------------
------- return self._query(self.root, self.min_x, self.max_x, x) -----
-------------------------------------------------------------------------
--- def _query(self, node, l, r, x): ---------------------------------
------- if not node: -----------------------------------------------
----------- return float('-inf') ---------------------------------------
------- if r - l <= 1: ---------------------------------------------
----------- return node.line.eval(x) -----------------------------------
-----------  ----------------------------------------------------------
------- mid = (l + r) // 2 -----------------------------------------
------- res = node.line.eval(x) ------------------------------------
-------  ------------------------------------------------------------
------- if x < mid: ------------------------------------------------
----------- res = max(res, self._query(node.left, l, mid, x)) --------
------- else: ------------------------------------------------------
----------- res = max(res, self._query(node.right, mid, r, x)) -------
-----------  ----------------------------------------------------------
------- return res -------------------------------------------------
```

2.2. Divide And Conquer.

```
INF = float('inf') ------------------------------------------------------
-------------------------------------------------------------------------
def solve_dp(g, k_L, k_R, n_L, n_R, dp, cost_func): ---------------------
--- if n_L > n_R: ----------------------------------------------------
------- return -----------------------------------------------------
-------------------------------------------------------------------------
--- n_M = (n_L + n_R) // 2 -------------------------------------------
--- best_k = -1 ------------------------------------------------------
--- dp[g][n_M] = INF -------------------------------------------------
-------------------------------------------------------------------------
--- for k in range(k_L, min(n_M, k_R) + 1): --------------------------
------- current_val = dp[g - 1][k] + cost_func(k + 1, n_M) ----------
------- if current_val < dp[g][n_M]: -------------------------------
----------- dp[g][n_M] = current_val -----------------------------------
----------- best_k = k -------------------------------------------------
-------------------------------------------------------------------------
--- solve_dp(g, k_L, best_k, n_L, n_M - 1, dp, cost_func) ------------
--- solve_dp(g, best_k, k_R, n_M + 1, n_R, dp, cost_func) ------------
```

3. Geometry

3.1. Angles And Rotations.

```
def angle(a, b, c): # angle formed by abc in radians: PI < x <= PI ---
--- return abs((arg(a-b) - arg(c-b)) % (2 * math.pi)) ----------------
def rotate(p, a, d): # rotate point a about pivot p CCW at d radians -
--- return p + (a - p) * complex(math.cos(d), math.sin(d)) -----------
```

3.2. Distances.

```
-------------------------------------------------------------------------
# Distance: Point to 2D Line (ax + by + c = 0) ----------------------
def dist_pt_line_eq(p, a, b, c): ----------------------------------
--- return abs(a*p.real + b*p.imag + c) / math.hypot(a, b) -----------
-------------------------------------------------------------------------
# Distance: Point to 2D Line through points a and b ------------------
def dist_pt_line(p, a, b): ----------------------------------------
--- cp = (p - a).conjugate() * (b - a) -------------------------------
--- return abs(cp.imag) / abs(b - a) ---------------------------------
-------------------------------------------------------------------------
# Distance: Point to 3D Plane (ax + by + cz + d = 0) ----------------
def dist_pt_plane(p, a, b, c, d): ---------------------------------
--- # math.hypot supports 3+ arguments in Python 3.8+ ----------------
--- return abs(a*p.x + b*p.y + c*p.z + d) / math.hypot(a,b,c) --------
-------------------------------------------------------------------------
# Distance: Shortest distance between 3D lines AB and CD -------------
def dist_line_3d(A, B, C, D): -------------------------------------
--- u, v, w = B - A, D - C, A - C ------------------------------------
--- a, b = u.dot(u), u.dot(v) ----------------------------------------
--- c, d, e = v.dot(v), u.dot(w), v.dot(w) ---------------------------
--- det = a * c - b * b ----------------------------------------------
--- if det < 1e-9: # Parallel lines ----------------------------------
------- s, t = 0.0, d / b if b > c else e / c ----------------------
--- else: # Skew lines -----------------------------------------------
------- s = (b * e - c * d) / det ----------------------------------
------- t = (a * e - b * d) / det ----------------------------------
--- p1 = P3(A.x + u.x*s, A.y + u.y*s, A.z + u.z*s) -------------------
--- p2 = P3(C.x + v.x*t, C.y + v.y*t, C.z + v.z*t) -------------------
--- vec = p1 - p2 ----------------------------------------------------
--- return math.sqrt(vec.dot(vec)) -----------------------------------
```

3.3. Intersections.

3.3.1. *Circle Line Intersection.*

```
EPS = 1e-9 --------------------------------------------------------------
def projLine(p, a, b): # Project point p onto line segment ab --------
--- v = b - a --------------------------------------------------------
--- scalar = ((p - a) * v.conjugate()).real / abs(v)**2 -------------
--- return a + v * scalar --------------------------------------------
-------------------------------------------------------------------------
def rotate(c, p, t): # Rotate point p around center c by t radians ---
--- return c + (p - c) * cmath.rect(1, t) ----------------------------
-------------------------------------------------------------------------
def CL_inter(c, r, a, b): -----------------------------------------
--- p = projLine(c, a, b) --------------------------------------------
--- d = abs(c - p) ---------------------------------------------------
--- ans = [] ---------------------------------------------------------
---  ----------------------------------------------------------------
--- if d > r + EPS: pass # None --------------------------------------
--- elif d > r - EPS: ans.append(p) # Tangent ------------------------
--- elif d < EPS: # Diameter -----------------------------------------
------- v = r * (b - a) / abs(b - a) -------------------------------
------- ans.append(c + v) ------------------------------------------
------- ans.append(c - v) ------------------------------------------
--- else: # Secant ---------------------------------------------------
------- t = math.acos(d / r) ---------------------------------------
------- p = c + (p - c) * r / d ------------------------------------
------- ans.append(rotate(c, p, t)) --------------------------------
------- ans.append(rotate(c, p, -t)) -------------------------------
--- return ans -------------------------------------------------------
```

3.3.2. *Circle Circle Intersection.*

```
EPS = 1e-9
def rotate(c, p, t):
    # Rotate point p around center c by t radians
    return c + (p - c) * cmath.rect(1, t)

def CC_inter(c1, r1, c2, r2):
    d = abs(c1 - c2)
    ans = []
    if d < EPS: # Concentric circles
        if abs(r1 - r2) < EPS:
            pass # Infinite intersections (same circle)
    elif r1 < EPS: # Circle 1 is a point
        if abs(d - r2) < EPS:
            ans.append(c1)
    else: # General case using Law of Cosines
        s = (r1*r1 + d*d - r2*r2) / (2*r1*d)
        s = max(-1.0, min(1.0, s)) # Prevent float domain errors
        t = math.acos(s)
        # Point on C1's radius pointing towards C2
        mid = c1 + (c2 - c1) * r1 / d
        ans.append(rotate(c1, mid, t))
        if abs(math.sin(t)) >= EPS:
            # FIXED: Rotate around c1, not c2!
            ans.append(rotate(c1, mid, -t))
    return ans
```

3.3.3. *Line Intersect.*

```
def line_inter_fast(a, b, c, d):
    A = cross(d - a, b - a)
    B = cross(c - a, b - a)
    return (B * d - A * c) / (B - A) if B - A else None
```

3.4. Closest Pair.

```
def closest_pair(pts):
    if len(pts) <= 1: return float('inf')
    px = sorted(pts) # Automatically sorts by x, then y
    def solve(l, r): # Base case: Brute force for small partitions
        if r - l <= 3:
            d = float('inf')
            for i in range(l, r):
                for j in range(i + 1, r):
                    d = min(d, math.dist(px[i], px[j]))
            return d
        mid = (l + r) // 2
        mid_x = px[mid][0]
        # Recursively find the minimum in both halves
        d = min(solve(l, mid), solve(mid, r))
        # Gather points in the middle strip
        strip = [p for p in px[l:r] if abs(p[0] - mid_x) < d]
        strip.sort(key=lambda p: p[1]) # Sort strip by y
        # Check at most 7 geometric neighbors in the strip
        for i, p1 in enumerate(strip):
            for p2 in strip[i+1 : i+8]:
                d = min(d, math.dist(p1, p2))
        return d
    return solve(0, len(px))
```

3.5. Convex Hull.

3.5.1. *Convex Hull.*

```
def cross(o, a, b): # 2D cross product of OA and OB vectors
    return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])

def convex_hull(pts): # Sorts by x, then by y automatically
    pts = sorted(set(pts))
    if len(pts) <= 1:
        return pts
    def build(iterable):
        h = []
        for p in iterable:
            while len(h) >= 2 and cross(h[-2], h[-1], p) <= 0:
                h.pop()
            h.append(p)
        return h
    lower = build(pts)
    upper = build(reversed(pts))
    return lower[:-1] + upper[:-1]
```

3.5.2. *Convex Hull 3D.*

```
class P3:
    __slots__ = ['x', 'y', 'z']
    def __init__(self, x=0, y=0, z=0):
        self.x, self.y, self.z = x, y, z
    def __sub__(self, o):
        return P3(self.x-o.x, self.y-o.y, self.z-o.z)
    def cross(self, o):
        return P3(self.y*o.z - self.z*o.y,
                  self.z*o.x - self.x*o.z,
                  self.x*o.y - self.y*o.x)
    def dot(self, o):
        return self.x*o.x + self.y*o.y + self.z*o.z
    def __eq__(self, o):
        return (self.x, self.y, self.z) == (o.x, o.y, o.z)
    def __lt__(self, o):
        return (self.x, self.y, self.z) < (o.x, o.y, o.z)

class Face:
    __slots__ = ['p', 'q']
    def __init__(self, p, q):
        self.p = p # Tuple of 3 point indices
        self.q = q # Normal vector (P3 object)

def hull_3D(pts):
    n = len(pts)
    faces = []
    dead = [[True] * n for _ in range(n)]
    def add_face(a, b, c):
        q = (pts[b] - pts[a]).cross(pts[c] - pts[a])
        faces.append(Face((a, b, c), q))
        dead[a][b] = dead[b][c] = dead[c][a] = False
    # Initialize with a degenerate tetrahedron
    add_face(0, 1, 2)
    add_face(0, 2, 1)
    for i in range(3, n):
        f_inv = []
        for f in faces:
            # If face is visible from the point
            if (pts[i] - pts[f.p[0]]).dot(f.q) > 0:
                for j in range(3):
                    dead[f.p[j]][f.p[(j+1)%3]] = True
            else:
                f_inv.append(f)
        faces = []
        for f in f_inv:
            for j in range(3):
                a, b = f.p[j], f.p[(j+1)%3]
                # If reverse edge is dead but forward is alive
                if dead[b][a]:
                    add_face(b, a, i)
        faces.extend(f_inv)
    return faces
```

3.6. Areas.

3.6.1. *Triangle Area.*

```
def area(a, b, c):
    s = (a + b + c) / 2
    return (s*(s-a)*(s-b)*(s-c)) ** 0.5
```

3.6.2. *Polygon Area.*

```
def area(p, n):
    a = 0
    for i in range(n):
        j = (i - 1) % n
        a += cross(p[i], p[j])
    return abs(a) / 2
```

3.7. Dot Cross.

```
def dot(a, b):
    return a.x * b.x + a.y * b.y # + a.z * b.z
def cross(a, b):
    return a.x * b.y - a.y * b.x
def cross(a, b, c):
    return cross(a, b) + cross(b, c) + cross(c, a)
def cross3D(a, b):
    return point3(a.x*b.y - a.y*b.x, a.y*b.z - a.z*b.y, a.z*b.x -
↪   a.x*b.z)
```

3.8. Line Sweep Closest Pair.

```
import math

def dist_sq(p1, p2):
    return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2

def closest_pair(points):
    points.sort()

    def solve(pts):
        n = len(pts)

        if n <= 3:
            return min(dist_sq(pts[i], pts[j])
                        for i in range(n)
                        for j in range(i + 1, n)) if n > 1 else
↪   float('inf')

        mid = n // 2
        mid_x = pts[mid][0]

        left = pts[:mid]
        right = pts[mid:]

        d = min(solve(left), solve(right))

        strip = [p for p in pts if (p[0] - mid_x) ** 2 < d]
        strip.sort(key=lambda p: p[1])

        for i in range(len(strip)):
            for j in range(i + 1, min(i + 7, len(strip))):
                d = min(d, dist_sq(strip[i], strip[j]))

        return d

    return solve(points)
```

4. Graphs

4.1. Max Flow.

4.1.1. *Dinic.*

```
INF = float('inf')
class Dinic:
    def __init__(self, n, s, t):
        self.n, self.s, self.t = n, s, t
        self.edges = []
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v, c, bi=False):
        self.adj[u].append(len(self.edges))
        self.edges.append([v, c, 0])
        self.adj[v].append(len(self.edges))
        self.edges.append([u, c if bi else 0, 0])

    def bfs(self):
        self.dist = [-1] * self.n
        self.dist[self.s] = 0
        q = deque([self.s])
        while q:
            u = q.popleft()
            for i in self.adj[u]:
                v, c, f = self.edges[i]
                if self.dist[v] < 0 and c > f:
                    self.dist[v] = self.dist[u] + 1
                    q.append(v)
        return self.dist[self.t] != -1

    def dfs(self, u, pushed):
        if pushed == 0 or u == self.t: return pushed
        while self.ptr[u] < len(self.adj[u]):
            i = self.adj[u][self.ptr[u]]
            v, c, f = self.edges[i]
            if self.dist[u]+1 == self.dist[v] and c > f:
                tr = self.dfs(v, min(pushed, c - f))
                if tr > 0:
                    self.edges[i][2] += tr
                    self.edges[i^1][2] -= tr
                    return tr
            self.ptr[u] += 1
        return 0

    def calc_max_flow(self):
        flow = 0
        while self.bfs():
            self.ptr = [0] * self.n
            while True:
                pushed = self.dfs(self.s, INF)
                if not pushed:
                    break
                flow += pushed
        return flow
```

4.1.2. *Edmonds Karp.*

```
INF = float('inf')
class EdmondsKarp:
    def __init__(self, n, s, t):
        self.n, self.s, self.t = n, s, t
        self.adj = [[] for _ in range(n)]
        self.res = [[0] * n for _ in range(n)]

    def add_edge(self, u, v, w):
        self.adj[u].append(v)
        self.adj[v].append(u)
        self.res[u][v] += w

    def bfs(self):
        self.par = [-1] * self.n
        self.par[self.s] = self.s
        q = deque([self.s])
        while q:
            u = q.popleft()
            for v in self.adj[u]:
                if self.res[u][v] > 0 and self.par[v] == -1:
                    self.par[v] = u
                    if v == self.t:
                        return True
                    q.append(v)
        return False

    def calc_max_flow(self):
        ans = 0
        while self.bfs():
            flow = INF
            u = self.t
            while u != self.s:
                p = self.par[u]
                flow = min(flow, self.res[p][u])
                u = p
            u = self.t
            while u != self.s:
                p = self.par[u]
                self.res[p][u] -= flow
                self.res[u][p] += flow
                u = p
            ans += flow
        return ans
```

4.1.3. *Bipartite Matching.*

```
from collections import deque

INF = float('inf')
class BipartiteGraph:
    def __init__(self, n, m):
        self.n = n
        self.m = m
        self.adj = [[] for _ in range(n)]
        self.L = [-1] * n
        self.R = [-1] * m
        self.dist = [INF] * (n + 1)

    def add_edge(self, u, v):
        self.adj[u].append(v)

    def bfs(self):
        queue = deque()
        for v in range(self.n):
            if self.L[v] == -1:
                self.dist[v] = 0
                queue.append(v)
            else:
                self.dist[v] = INF
        self.dist[self.n] = INF
        while queue:
            v = queue.popleft()
            if self.dist[v] < self.dist[self.n]:
                for u in self.adj[v]:
                    next_v = self.R[u]
                    dist_next = self.dist[next_v] if next_v != -1 else
↪   self.dist[self.n]
                    if (next_v == -1 and self.dist[self.n] == INF) or \
                        (next_v != -1 and self.dist[next_v] ==
↪   INF):
                        target = next_v if next_v != -1 else self.n
                        self.dist[target] = self.dist[v] + 1
                        if target != self.n:
                            queue.append(target)
        return self.dist[self.n] != INF

    def dfs(self, v):
        if v != -1:
            for u in self.adj[v]:
                next_v = self.R[u]
                dist_next = self.dist[next_v] if next_v != -1 else
↪   self.dist[self.n]
                if dist_next == self.dist[v] + 1:
                    if self.dfs(next_v):
                        self.R[u] = v
                        self.L[v] = u
                        return True
            self.dist[v] = INF
            return False
        return True

    def maximum_matching(self):
        matching = 0
        for u in range(self.n):
            for v in self.adj[u]:
                if self.R[v] == -1:
                    self.L[u] = v
                    self.R[v] = u
                    matching += 1
                    break
        while self.bfs():
            for i in range(self.n):
                if self.L[i] == -1 and self.dfs(i):
                    matching += 1
        return matching
# Input Format: g = BipartiteGraph(2, 2)
#               g.add_edge(0, 0)
#               g.add_edge(0, 1)
#               g.add_edge(1, 0)
#               print(f"Max Matching: {g.maximum_matching()}")
```

4.2. Shortest Path.

4.2.1. *Dijkstra.*

```
import heapq
INF = float('inf')
def dijkstra(n, adj, s):
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]

    while pq:
        d, u = heapq.heappop(pq)
        if dist[u] < d:
            continue
        for v, w in adj[u]:
            if dist[v] > d + w:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

4.2.2. *Bellman Ford.*

```
INF = float('inf')
def bellman_ford(n, adj, s):
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):
        for u in range(n):
            if dist[u] == INF: continue
            for v, w in adj[u]:
                if dist[u] + w < dist[v]:
                    dist[v] = dist[u] + w
    for u in range(n):
```

```
------- if dist[u] == INF: continue ----------------------------------
------- for v, w in adj[u]: ------------------------------------------
----------- if dist[u] + w < dist[v]: ------------------------------
--------------- return dist, True ------------------------------------
--- return dist, False -------------------------------------------------
```

4.3. Minimum Spanning Tree.

4.3.1. *Kruskal Min Spanning Tree.*

```
import heapq -------------------------------------------------------------

class UnionFind: ---------------------------------------------------------
--- def __init__(self, n): ---------------------------------------------
------- self.parent = list(range(n)) -------------------------------------
------- self.rank = [0] * n ----------------------------------------------

--- def find(self, x): -------------------------------------------------
------- while self.parent[x] != x: ---------------------------------------
----------- self.parent[x] = self.parent[self.parent[x]] -------------
----------- x = self.parent[x] -----------------------------------------
------- return x ---------------------------------------------------------

--- def unite(self, a, b): ---------------------------------------------
------- a = self.find(a) -------------------------------------------------
------- b = self.find(b) -------------------------------------------------

------- if a == b: -------------------------------------------------------
----------- return False -----------------------------------------------

------- if self.rank[a] < self.rank[b]: ----------------------------------
----------- self.parent[a] = b -----------------------------------------
------- else: ------------------------------------------------------------
----------- self.parent[b] = a -----------------------------------------
----------- if self.rank[a] == self.rank[b]: ---------------------------
--------------- self.rank[a] += 1 ------------------------------------

------- return True ------------------------------------------------------

def kruskal(n, edges): ---------------------------------------------------
--- edges = edges[:] ---------------------------------------------------
--- heapq.heapify(edges) -----------------------------------------------

--- uf = UnionFind(n) --------------------------------------------------
--- total_weight = 0 ---------------------------------------------------
--- mst = [] -----------------------------------------------------------

--- while edges: -------------------------------------------------------
------- w, u, v = heapq.heappop(edges) -----------------------------------
------- if uf.unite(u, v): -----------------------------------------------
----------- mst.append((u, v, w)) --------------------------------------
----------- total_weight += w ------------------------------------------

----------- if len(mst) == n - 1: --------------------------------------
--------------- break ------------------------------------------------

--- return mst, total_weight -------------------------------------------
```

4.3.2. *Prim Min Spanning Tree.*

```
import heapq -------------------------------------------------------------

def prim(n, adj, start=0): -----------------------------------------------
--- visited = [False] * n ----------------------------------------------
--- pq = [(0, start, -1)] ----------------------------------------------

--- total_weight = 0 ---------------------------------------------------
--- mst = [] -----------------------------------------------------------

--- while pq: ----------------------------------------------------------
------- w, u, parent = heapq.heappop(pq) ---------------------------------

------- if visited[u]: ---------------------------------------------------
----------- continue ---------------------------------------------------

------- visited[u] = True ------------------------------------------------

------- if parent != -1: -------------------------------------------------
----------- mst.append((parent, u, w)) ---------------------------------
----------- total_weight += w ------------------------------------------

------- for v, weight in adj[u]: -----------------------------------------
----------- if not visited[v]: -----------------------------------------
--------------- heapq.heappush(pq, (weight, v, u)) -------------------

--- return mst, total_weight -------------------------------------------
```

4.4. Least Common Ancestor.

4.4.1. *Binary Lifting Lca.*

```
import math ----------------------------------------------------------------

class BinaryLifting: -----------------------------------------------------
--- def __init__(self, adj): -------------------------------------------
------- self.n = len(adj) ------------------------------------------------
------- self.LOG = math.ceil(math.log2(self.n)) + 1 ----------------------
------- self.adj = adj ---------------------------------------------------

------- self.dep = [0] * self.n ------------------------------------------
------- self.par = [[-1] * self.LOG for _ in range(self.n)] --------------

------- self.build(0) ----------------------------------------------------

--- def dfs(self, u, p): -----------------------------------------------
------- self.par[u][0] = p -----------------------------------------------
------- for v in self.adj[u]: --------------------------------------------
----------- if v != p: -------------------------------------------------
--------------- self.dep[v] = self.dep[u] + 1 ------------------------
--------------- self.dfs(v, u) ---------------------------------------

--- def build(self, root): ---------------------------------------------
------- self.dfs(root, root) ---------------------------------------------

------- for k in range(1, self.LOG): -------------------------------------
----------- for u in range(self.n): ------------------------------------
--------------- self.par[u][k] = self.par[self.par[u][k-1]][k-1] -----

--- def lift(self, u, k): ----------------------------------------------
------- for i in range(self.LOG): ----------------------------------------
----------- if k & (1 << i): -------------------------------------------
--------------- u = self.par[u][i] -----------------------------------
------- return u ---------------------------------------------------------

--- def lca(self, u, v): -----------------------------------------------
------- if self.dep[u] < self.dep[v]: ------------------------------------
----------- u, v = v, u ------------------------------------------------

------- u = self.lift(u, self.dep[u] - self.dep[v]) ----------------------

------- if u == v: -------------------------------------------------------
----------- return u ---------------------------------------------------

------- for k in reversed(range(self.LOG)): ------------------------------
----------- if self.par[u][k] != self.par[v][k]: -----------------------
--------------- u = self.par[u][k] -----------------------------------
--------------- v = self.par[v][k] -----------------------------------

------- return self.par[u][0] --------------------------------------------
```

4.4.2. *Euler Lca.*

```
class EulerLCA: ----------------------------------------------------------
--- def __init__(self, adj): -------------------------------------------
------- self.n = len(adj) ------------------------------------------------
------- self.adj = adj ---------------------------------------------------

------- self.euler = [] --------------------------------------------------
------- self.first = [-1] * self.n ---------------------------------------
------- self.depth = [0] * self.n ----------------------------------------

------- self.build(0) ----------------------------------------------------

--- def dfs(self, u, p, d): --------------------------------------------
------- self.first[u] = len(self.euler) ----------------------------------
------- self.depth[u] = d ------------------------------------------------
------- self.euler.append(u) ---------------------------------------------

------- for v in self.adj[u]: --------------------------------------------
----------- if v != p: -------------------------------------------------
--------------- self.dfs(v, u, d + 1) --------------------------------
--------------- self.euler.append(u) ---------------------------------

--- def build(self, root): ---------------------------------------------
------- self.dfs(root, root, 0) ------------------------------------------

------- m = len(self.euler) ----------------------------------------------
------- LOG = m.bit_length() ---------------------------------------------

------- self.st = [[0] * m for _ in range(LOG)] --------------------------
------- self.log = [0] * (m + 1) -----------------------------------------

------- for i in range(2, m + 1): ----------------------------------------
----------- self.log[i] = self.log[i // 2] + 1 -------------------------

------- for i in range(m): -----------------------------------------------
----------- self.st[0][i] = self.euler[i] ------------------------------

------- for k in range(1, LOG): ------------------------------------------
----------- for i in range(m - (1 << k) + 1): --------------------------
--------------- a = self.st[k-1][i] ----------------------------------
--------------- b = self.st[k-1][i + (1 << (k-1))] -------------------
--------------- self.st[k][i] = a if self.depth[a] < self.depth[b] else
↪  b

--- def lca(self, u, v): -----------------------------------------------
------- l, r = self.first[u], self.first[v] ------------------------------
------- if l > r: --------------------------------------------------------
----------- l, r = r, l ------------------------------------------------

------- k = self.log[r - l + 1] ------------------------------------------
------- a = self.st[k][l] ------------------------------------------------
------- b = self.st[k][r - (1 << k) + 1] ---------------------------------

------- return a if self.depth[a] < self.depth[b] else b -----------------
```

4.4.3. *Tarjan Lca.*

```
class DSU: ---------------------------------------------------------------
--- def __init__(self, n): ---------------------------------------------
------- self.p = list(range(n)) ------------------------------------------

--- def find(self, x): -------------------------------------------------
------- if self.p[x] != x: -----------------------------------------------
----------- self.p[x] = self.find(self.p[x]) ---------------------------
------- return self.p[x] -------------------------------------------------

--- def unite(self, a, b): ---------------------------------------------
------- self.p[self.find(b)] = self.find(a) ------------------------------

class Tarjan: ------------------------------------------------------------
--- def __init__(self, adj, queries): ----------------------------------
------- self.adj = adj ---------------------------------------------------
------- self.queries = queries -------------------------------------------
------- self.n = len(adj) ------------------------------------------------
```

```
        self.dsu = DSU(self.n)
        self.vis = [False] * self.n
        self.anc = [0] * self.n
        self.res = {}

    def dfs(self, u, p):
        self.anc[u] = u

        for v in self.adj[u]:
            if v == p:
                continue
            self.dfs(v, u)
            self.dsu.unite(u, v)
            self.anc[self.dsu.find(u)] = u

        self.vis[u] = True

        for v, idx in self.queries[u]:
            if self.vis[v]:
                self.res[idx] = self.anc[self.dsu.find(v)]
```

4.5. Minimum Mean Weight Cycle.

```
def min_mean_cycle(adj):
    n = len(adj)
    INF = float('inf')

    arr = [[INF] * n for _ in range(n + 1)]

    for i in range(n):
        arr[0][i] = 0

    for k in range(1, n + 1):
        for u in range(n):
            if arr[k - 1][u] == INF:
                continue
            for v, w in adj[u]:
                arr[k][v] = min(arr[k][v], arr[k - 1][u] + w)

    result = INF

    for v in range(n):
        if arr[n][v] == INF:
            continue

        max_avg = -INF
        for k in range(n):
            if arr[k][v] == INF:
                continue
            avg = (arr[n][v] - arr[k][v]) / (n - k)
            max_avg = max(max_avg, avg)

        result = min(result, max_avg)

    return result
```

4.6. Strongly Connected Components.

```
class KosarajuGraph:
    def __init__(self, n):
        self.n = n
        self.graph = [[] for _ in range(n)]
        self.rev_graph = [[] for _ in range(n)]
        self.vis = [0] * n
        self.sccs = []

    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.rev_graph[v].append(u)

    def dfs(self, u, adj, container):
        self.vis[u] = 1
        for v in adj[u]:
            if not self.vis[v]:
                self.dfs(v, adj, container)
        container.append(u)

    def kosaraju(self):
        topo = []

        # First pass
        self.vis = [0] * self.n
        for i in range(self.n):
            if not self.vis[i]:
                self.dfs(i, self.graph, topo)

        # Second pass
        self.vis = [0] * self.n
        self.sccs = []

        for u in reversed(topo):
            if not self.vis[u]:
                comp = []
                self.dfs(u, self.rev_graph, comp)
                self.sccs.append(comp)

        return self.sccs
```

4.7. Heavy Light Decomposition.

```
class SegmentTree:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (4 * n)

    # Example: point update
    def update(self, idx, val, node=1, l=0, r=None):
        if r is None:
            r = self.n - 1
        if l == r:
            self.tree[node] = vala
            return
        mid = (l + r) // 2
        if idx <= mid:
            self.update(idx, val, node * 2, l, mid)
        else:
            self.update(idx, val, node * 2 + 1, mid + 1, r)
        self.tree[node] = self.tree[node * 2] + self.tree[node * 2 + 1]

    # Example: range query
    def query(self, ql, qr, node=1, l=0, r=None):
        if r is None:
            r = self.n - 1
        if qr < l or r < ql:
            return 0
        if ql <= l and r <= qr:
            return self.tree[node]
        mid = (l + r) // 2
        return self.query(ql, qr, node * 2, l, mid) + \
               self.query(ql, qr, node * 2 + 1, mid + 1, r)

class HeavyLightTree:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]

        self.par = [-1] * n
        self.heavy = [-1] * n
        self.dep = [0] * n
        self.path_root = [0] * n
        self.pos = [0] * n

        self.segment_tree = SegmentTree(n)
        self.current_pos = 0

    def add_edge(self, u, v):
        self.adj[u].append(v)
        self.adj[v].append(u)

    def dfs(self, u):
        size = 1
        max_subtree_size = 0

        for v in self.adj[u]:
            if v != self.par[u]:
                self.par[v] = u
                self.dep[v] = self.dep[u] + 1
                subtree_size = self.dfs(v)

                if subtree_size > max_subtree_size:
                    max_subtree_size = subtree_size
                    self.heavy[u] = v

                size += subtree_size

        return size

    def decompose(self, u, root):
        self.path_root[u] = root
        self.pos[u] = self.current_pos
        self.current_pos += 1

        if self.heavy[u] != -1:
            self.decompose(self.heavy[u], root)

        for v in self.adj[u]:
            if v != self.par[u] and v != self.heavy[u]:
                self.decompose(v, v)

    def build(self, root=0):
        self.par[root] = root
        self.dep[root] = 0
        self.dfs(root)
        self.decompose(root, root)

    # Query path u -> v
    def query(self, u, v):
        res = 0
        while self.path_root[u] != self.path_root[v]:
            if self.dep[self.path_root[u]] <
↪   self.dep[self.path_root[v]]:
                u, v = v, u
            start = self.path_root[u]
            res += self.segment_tree.query(self.pos[start],
↪   self.pos[u])
            u = self.par[start]

        # Same path
        if self.dep[u] > self.dep[v]:
            u, v = v, u
        res += self.segment_tree.query(self.pos[u], self.pos[v])
        return res

    # Update node value
    def update(self, u, value):
        self.segment_tree.update(self.pos[u], value)
```

4.8. Euler Path/Cycle.

4.8.1. *Directed Euler.*

```
from collections import defaultdict

class DirectedEuler:
    def __init__(self, n):
        self.n = n
        self.adj = defaultdict(list)
        self.indeg = [0]*n
        self.outdeg = [0]*n
        self.edge_count = 0

    def add_edge(self, u, v):
        self.adj[u].append(v)
        self.outdeg[u] += 1
        self.indeg[v] += 1
        self.edge_count += 1

    def find_start_node(self):
        start = -1
        end = -1
        for i in range(self.n):
            if self.outdeg[i] - self.indeg[i] == 1:
                if start != -1:
                    return -1
                start = i
            elif self.indeg[i] - self.outdeg[i] == 1:
                if end != -1:
                    return -1
                end = i
            elif self.indeg[i] != self.outdeg[i]:
                return -1

        if start == -1:
            for i in range(self.n):
                if self.outdeg[i] > 0:
                    return i
        return start

    def get_euler_path(self):
        start = self.find_start_node()
        if start == -1:
            return None

        stack = [start]
        path = []

        local_adj = {u: self.adj[u][:] for u in self.adj}

        while stack:
            u = stack[-1]
            if u in local_adj and local_adj[u]:
                v = local_adj[u].pop()
                stack.append(v)
            else:
                path.append(stack.pop())

        if len(path) != self.edge_count + 1:
            return None

        return path[::-1]


    class UndirectedEuler:
        def __init__(self, n):
            self.n = n
            self.adj = defaultdict(list)
            self.edge_count = 0

        def add_edge(self, u, v):
            self.adj[u].append(v)
            self.adj[v].append(u)
            self.edge_count += 1

        def is_valid(self):
            odd = 0
            for i in range(self.n):
                if len(self.adj[i]) % 2 != 0:
                    odd += 1
            return odd == 0 or odd == 2

        def get_euler_path(self):
            if not self.is_valid():
                return None

            start = 0
            for i in range(self.n):
                if len(self.adj[i]) % 2 == 1:
                    start = i
                    break
                if len(self.adj[i]) > 0:
                    start = i

            stack = [start]
            path = []

            local_adj = {u: self.adj[u][:] for u in self.adj}

            while stack:
                u = stack[-1]
                if u in local_adj and local_adj[u]:
                    v = local_adj[u].pop()
                    local_adj[v].remove(u)
                    stack.append(v)
                else:
                    path.append(stack.pop())

            if len(path) != self.edge_count + 1:
                return None

            return path[::-1]
```

4.8.2. *Undirected Euler.*

```
from collections import defaultdict

class UndirectedEuler:
    def __init__(self, n):
        self.n = n
        self.adj = defaultdict(list)
        self.edge_count = 0

    def add_edge(self, u, v):
        self.adj[u].append(v)
        self.adj[v].append(u)
        self.edge_count += 1

    def is_valid(self):
        odd = 0
        for i in range(self.n):
            if len(self.adj[i]) % 2 != 0:
                odd += 1
        return odd == 0 or odd == 2

    def get_euler_path(self):
        if not self.is_valid():
            return None

        start = 0
        for i in range(self.n):
            if len(self.adj[i]) % 2 == 1:
                start = i
                break
            if len(self.adj[i]) > 0:
                start = i

        stack = [start]
        path = []

        local_adj = {u: self.adj[u][:] for u in self.adj}

        while stack:
            u = stack[-1]
            if u in local_adj and local_adj[u]:
                v = local_adj[u].pop()
                local_adj[v].remove(u)
                stack.append(v)
            else:
                path.append(stack.pop())

        if len(path) != self.edge_count + 1:
            return None

        return path[::-1]
```

4.9. Tree Isomorphism.

```
from collections import deque

def bfs(u, adj, n):
    vis = [False] * n
    pre = [-1] * n
    q = deque()

    q.append(u)
    vis[u] = True

    last = u

    while q:
        u = q.popleft()
        last = u

        for v in adj[u]:
            if not vis[v]:
                vis[v] = True
                pre[v] = u
                q.append(v)

    return last, pre

def tree_centers(r, adj, n):
    u, _ = bfs(r, adj, n)
    v, pre = bfs(u, adj, n)

    path = []
    while v != -1:
        path.append(v)
        v = pre[v]

    size = len(path)

    med = [path[size // 2]]
    if size % 2 == 0:
        med.append(path[size // 2 - 1])

    return med

def rootcode(u, adj, p, d, pr):
    children = []
    for v in adj[u]:
        if v != p:
            children.append(rootcode(v, adj, u, (d + 1) % len(pr), pr))

    children.sort()
```

```
    h = len(children) + 1
    for x in children:
        h = h * pr[d] + x

    return h

def treecode(root, adj, n, pr):
    c = tree_centers(root, adj, n)

    if len(c) == 1:
        return (rootcode(c[0], adj, -1, 0, pr) << 1) | 1
    else:
        return (rootcode(c[0], adj, -1, 0, pr) *
                rootcode(c[1], adj, -1, 0, pr)) << 1

def isomorphic(r1, adj1, r2, adj2, n1, n2, pr, rooted=False):
    if rooted:
        return rootcode(r1, adj1, -1, 0, pr) == rootcode(r2, adj2, -1,
↪  0, pr)
    return treecode(r1, adj1, n1, pr) == treecode(r2, adj2, n2, pr) --
```

4.10. Centroid Decomposition.

```
import sys
sys.setrecursionlimit(10**7)

INF = 10**9

class CentroidDecomposition:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]

        self.LG = 20
        self.jmp = [[-1] * self.LG for _ in range(n)]
        self.path = [[INF] * self.LG for _ in range(n)]

        self.sz = [0] * n
        self.seph = [0] * n
        self.shortest = [INF] * n
        self.blocked = [False] * n

    def add_edge(self, a, b):
        self.adj[a].append(b)
        self.adj[b].append(a)

    def dfs(self, u, p):
        self.sz[u] = 1
        for v in self.adj[u]:
            if v != p and not self.blocked[v]:
                self.sz[u] += self.dfs(v, u)
        return self.sz[u]

    def find_centroid(self, u, p, n):
        for v in self.adj[u]:
            if v != p and not self.blocked[v] and self.sz[v] > n // 2:
                return self.find_centroid(v, u, n)
        return u

    def makepaths(self, sep, u, p, depth, h):
        self.jmp[u][h] = sep
        self.path[u][h] = depth

        for v in self.adj[u]:
            if v != p and not self.blocked[v]:
                self.makepaths(sep, v, u, depth + 1, h)

    def separate(self, h=0, u=0):
        self.dfs(u, -1)
        c = self.find_centroid(u, -1, self.sz[u])

        self.seph[c] = h
        self.makepaths(c, c, -1, 0, h)

        self.blocked[c] = True

        for v in self.adj[c]:
            if not self.blocked[v]:
                self.separate(h + 1, v)

    def paint(self, u):
        for h in range(self.seph[u] + 1):
            c = self.jmp[u][h]
            self.shortest[c] = min(self.shortest[c], self.path[u][h])

    def closest(self, u):
        ans = INF
        for h in range(self.seph[u] + 1):
            c = self.jmp[u][h]
            ans = min(ans, self.shortest[c] + self.path[u][h])
        return ans
```

5. Math I Algebra

6. Math II Combinatorics

6.0.1. *Lucas Theorem.*

```
_fact_cache = []
_last_p = -1

def lucas(n, k, p):
    if k < 0 or k > n:
        return 0
    if k == 0 or n == k:
        return 1

    global _fact_cache, _last_p
    if _last_p != p:
        _fact_cache = [1] * p
        for i in range(2, p):
            _fact_cache[i] = (_fact_cache[i - 1] * i) % p
        _last_p = p

    res_num = 1
    res_den = 1

    while n > 0 or k > 0:
        ni, ki = n % p, k % p
        if ki > ni:
            return 0
        res_num = (res_num * _fact_cache[ni]) % p
        res_den = (res_den * _fact_cache[ki] * _fact_cache[ni - ki]) %
↪  p
        n //= p
        k //= p
    return (res_num * pow(res_den, p - 2, p)) % p
# Input Format: print(lucas(n, k, p))
```

6.0.2. *Catalan Numbers.*

```
def catalan(n):
    if (n == 0 or n == 1):
        return 1

    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1

    for i in range(2, n + 1):
        for j in range(i):
            dp[i] += dp[j] * dp[i-j-1]

    return dp[n]

# for i in range(10):
#     print(catalan(i))
```

6.0.3. *Kth Permutation.*

```
#fact = [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880]

def findKthPermutation(n, k):
    nums = [str(i) for i in range(1, n + 1)]

    fact = [1] * n
    for i in range(1, n):
        fact[i] = fact[i - 1] * i

    k -= 1
    res = []

    for i in range(n - 1, -1, -1):
        idx = k // fact[i]
        k %= fact[i]

        res.append(nums.pop(idx))

    return ''.join(res)
```

6.0.4. *Factoradics.*

```
def factoradic(arr):
    n = len(arr)
    ft = FenwickTree(n)
    for i in range(1, n + 1):
        ft.add(i, 1)

    res = []
    for val in arr:
        bit_val = val + 1
        s = ft.query(bit_val - 1)
        ft.add(bit_val, -1)
        res.append(s)
    return res

def permute(fact, n):
    ft = FenwickTree(n)
    for i in range(1, n + 1):
        ft.add(i, 1)

    res = []
    for val in fact:
        idx = ft.find_kth(val)
        ft.add(idx, -1)
        res.append(idx - 1)
    return res
```

7. Math III Number Theory

7.0.1. *Gcd Subset Counter.*

```
def gcd_subset_count(arr):
    n = len(arr)
    mx = max(arr)

    f = [0] * (mx + 1)
    for x in arr:
        f[x] += 1

    gcnt = [0] * (mx + 1)

    for i in range(mx, 0, -1):
        add = 0

        for j in range(i, mx + 1, i):
            add += f[j]
```

```
    total = (1 << add) - 1  # 2^add - 1

    sub = 0
    for j in range(2 * i, mx + 1, i):
        sub += gcnt[j]

    gcnt[i] = total - sub

return gcnt
```

7.0.2. *Linear Diophantine.*

```
def gcd(a, b):
    if a == 0:
        return b
    return gcd(b % a, a)

def isPossible(a, b, c):
    g = gcd(a, b)

    return c % g == 0
```

7.0.3. *Chinese Remainder Theorem.*

```
def gcd_extended(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = gcd_extended(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def find_min_x(num, rem):
    prod = 1
    for n in num:
        prod *= n

    result = 0
    for i in range(len(num)):
        prod_i = prod // num[i]
        _, inv_i, _ = gcd_extended(prod_i, num[i])
        result += rem[i] * prod_i * inv_i

    return result % prod
```

8. Math IV Numerical Methods

8.0.1. *Fast Square Testing.*

```
import math

class FastSquareTester:
    def __init__(self):
        self.M = 0
        self.init_is_square()

    def init_is_square(self):
        for i in range(64):
            self.M |= (1 << (i * i % 64))

    def is_square(self, x):
        if x < 0: return False
        if x == 0: return True

        if not (self.M & (1 << (x & 63))):
            return False

        c = (x & -x).bit_length() - 1
        if c & 1:
            return False

        temp_x = x >> c
        if (temp_x & 7) != 1:
            return False

        r = int(math.isqrt(temp_x))
        return r * r == temp_x

tester = FastSquareTester()
```

8.0.2. *Simpson Integration.*

```
def f(x):
    return x ** 2
def simpson_integration(a, b, n=1000000):
    if n % 2 != 0:
        n += 1

    h = (b - a) / n
    s = f(a) + f(b)

    for i in range(1, n):
        x = a + h * i
        if i % 2 == 1:
            s += 4 * f(x)
        else:
            s += 2 * f(x)

    return s * (h / 3)
```

9. Strings

9.0.1. *Longest Common Prefix.*

```
def build_lcp(s, sa):
    n = len(s)
    rank = [0] * n
    lcp = [0] * n

    for i in range(n):
        rank[sa[i]] = i

    k = 0
    for i in range(n):
        if rank[i] == 0:
            k = 0
            continue

        j = sa[rank[i] - 1]

        while i + k < n and j + k < n and s[i + k] == s[j + k]:
            k += 1

        lcp[rank[i]] = k

        if k > 0:
            k -= 1

    return lcp
```

9.0.2. *Hasher.*

```
class Hasher:
    def __init__(self, s, bases=[31, 37], mod=10 ** 9 + 7):
        self.bases = bases
        self.mod = mod
        self.n = len(s)
        self.num_hashes = len(bases)
        self.p_pow = [[1] * (self.n + 1) for _ in
↪   range(self.num_hashes)]
        self.h_ans = [[0] * (self.n + 1) for _ in
↪   range(self.num_hashes)]

        for i, b in enumerate(self.bases):
            for j in range(self.n):
                self.p_pow[i][j + 1] = (self.p_pow[i][j] * b) %
↪   self.mod
                self.h_ans[i][j + 1] = (self.h_ans[i][j] + s[j] *
↪   self.p_pow[i][j]) % self.mod

    def get_hash(self, l, r):
        return tuple((self.h_ans[i][r + 1] - self.h_ans[i][l]) %
↪   self.mod
                     for i in range(self.num_hashes))

    def are_equal(self, l1, r1, l2, r2):
        if (r1 - l1) != (r2 - l2): return False

        h1 = self.get_hash(l1, r1)
        h2 = self.get_hash(l2, r2)

        return all((h1[i] * self.p_pow[i][l2]) % self.mod ==
                   (h2[i] * self.p_pow[i][l1]) % self.mod
                   for i in range(self.num_hashes))
```

9.0.3. *Z Algorithm.*

9.0.4. *Aho Corasick Trie.*

```
from collections import deque
import random
import string

class Node:
    def __init__(self):
        self.next = {}
        self.fail = None
        self.count = 0

    def add(self, s):
        node = self
        for c in s:
            if c not in node.next:
                node.next[c] = Node()
            node = node.next[c]
        node.count += 1

    def prepare(self):
        root = self
        q = deque()

        root.fail = root

        for child in root.next.values():
            child.fail = root
            q.append(child)

        while q:
            head = q.popleft()

            for c, nxt in head.next.items():
                f = head.fail

                while f != root and c not in f.next:
                    f = f.fail

                nxt.fail = f.next[c] if c in f.next else root

                nxt.count += nxt.fail.count

                q.append(nxt)

    def search(self, s):
        root = self
```

```
        p = self
        ans = 0

    for c in s:
        while p != root and c not in p.next:
            p = p.fail

        if c in p.next:
            p = p.next[c]
            ans += p.count

    return ans
```

9.0.5. *Knuth Morris Pratt.*

```
def build_kmp(f):
    n = len(f)
    par = [0] * (n + 1)

    par[0] = -1
    if n > 0:
        par[1] = 0

    i, j = 2, 0

    while i <= n:
        if f[i - 1] == f[j]:
            j += 1
            par[i] = j
            i += 1
        elif j > 0:
            j = par[j]
        else:
            par[i] = 0
            i += 1

    return par


def kmp(s, f):
    par = build_kmp(f)

    i = 0
    j = 0
    ans = []

    n, m = len(s), len(f)

    while i + j < n:
        if s[i + j] == f[j]:
            j += 1
            if j == m:
                ans.append(i)
                i += j - par[j]
                if j > 0:
                    j = par[j]
        else:
            i += j - par[j]
            if j > 0:
                j = par[j]

    return ans
```

9.0.6. *Trie.*

```
class Node:
    def __init__(self):
        self.children = {}
        self.prefixes = 0
        self.words = 0

class Trie:
    def __init__(self):
        self.root = Node()

    def insert(self, seq):
        cur = self.root

        for i in range(len(seq) + 1):
            cur.prefixes += 1

            if i == len(seq):
                cur.words += 1
                break

            head = seq[i]

            if head not in cur.children:
                cur.children[head] = Node()

            cur = cur.children[head]

    def countMatches(self, seq):
        cur = self.root

        for c in seq:
            if c not in cur.children:
                return 0
            cur = cur.children[c]

        return cur.words

    def countPrefixes(self, seq):
        cur = self.root

        for c in seq:
            if c not in cur.children:
                return 0
            cur = cur.children[c]

        return cur.prefixes
```

9.0.7. *Suffix Array.*

```
class SuffixArray:
    def __init__(self):
        self.n = 0
        self.suffix = []
        self.equiv = []
        self.equiv_pair = []
        self.s = ""

    def build(self, s):
        if not s.endswith('$'):
            s += '$'

        self.s = s
        self.n = len(s)

        self.suffix = list(range(self.n))
        self.equiv = [0] * self.n
        self.equiv_pair = [(0, 0)] * self.n

        self.suffix.sort(key=lambda i: s[i])

        cls = 0
        for i in range(self.n):
            if i == 0 or s[self.suffix[i]] != s[self.suffix[i - 1]]:
                cls += 1
            self.equiv[self.suffix[i]] = cls

        k = 1
        while k < self.n:
            for i in range(self.n):
                self.equiv_pair[i] = (
                    self.equiv[i],
                    self.equiv[(i + k) % self.n]
                )

            self.suffix.sort(key=lambda i: self.equiv_pair[i])

            cls = 0
            new_equiv = [0] * self.n

            for i in range(self.n):
                if i == 0 or self.equiv_pair[self.suffix[i]] !=
↪  self.equiv_pair[self.suffix[i - 1]]:
                    cls += 1
                new_equiv[self.suffix[i]] = cls

            self.equiv = new_equiv
            k <<= 1

    def _cmp(self, start, pattern):
        s = self.s
        n = len(s)

        i = start
        j = 0

        while i < n and j < len(pattern):
            if s[i] != pattern[j]:
                return ord(s[i]) - ord(pattern[j])
            i += 1
            j += 1

        if j == len(pattern):
            return 0
        return -1

    def lower_bound(self, pattern):
        l, r = 0, self.n - 1
        res = self.n

        while l <= r:
            m = (l + r) // 2
            if self._cmp(self.suffix[m], pattern) >= 0:
                res = m
                r = m - 1
            else:
                l = m + 1

        return res

    def upper_bound(self, pattern):
        l, r = 0, self.n - 1
        res = self.n

        while l <= r:
            m = (l + r) // 2
            if self._cmp(self.suffix[m], pattern) > 0:
                res = m
                r = m - 1
            else:
                l = m + 1

        return res

    def count_occurrences(self, pattern):
        L = self.lower_bound(pattern)
        R = self.upper_bound(pattern)
        return max(0, R - L)
```

9.0.8. *Booth Min String Rotation.*

```
def booth(s: str) -> int:
    s = s + s
    n = len(s)

    f = [-1] * n
    k = 0

    for j in range(1, n):
        i = f[j - k - 1]

        while i != -1 and s[j] != s[k + i + 1]:
            if s[j] < s[k + i + 1]:
                k = j - i - 1
            i = f[i]

        if i == -1 and s[j] != s[k]:
            if s[j] < s[k]:
                k = j
            f[j - k] = -1
        else:
            f[j - k] = i + 1

    return k
```

10. Other Algorithms

10.0.1. *Longest Increasing Subsequence.*

```
def lis(arr):
    if not arr:
        return []

    n = len(arr)
    seq = []
    back = [0] * n

    for i in range(n):
        lo, hi = 0, len(seq)
        while lo < hi:
            mid = (lo + hi) // 2
            if arr[seq[mid]] < arr[i]:
                lo = mid + 1
            else:
                hi = mid

        res = lo
        if res < len(seq):
            seq[res] = i
        else:
            seq.append(i)
        back[i] = -1 if res == 0 else seq[res - 1]

    ans = []
    at = seq[-1]
    while at != -1:
        ans.append(at)
        at = back[at]

    return ans[::-1]
```

10.0.2. *Cycle Find.*

```
def find_cycle(x0, f):
    t = f(x0)
    h = f(f(x0))

    while t != h:
        t = f(t)
        h = f(f(h))

    mu = 0
    h = x0
    while t != h:
        t = f(t)
        h = f(h)
        mu += 1

    lam = 1
    h = f(t)
    while t != h:
        h = f(h)
        lam += 1

    return mu, lam
```

10.0.3. *2Sat.*

```
class TwoSAT:
    def __init__(self, n):
        self.n = n
        self.N = 2 * n
        self.adj = [[] for _ in range(self.N)]

    def var(self, x):
        return x << 1

    def neg(self, x):
        return x ^ 1

    def add_imp(self, x, y):
        self.adj[x].append(y)

    def add_or(self, x, y):
        self.add_imp(self.neg(x), y)
        self.add_imp(self.neg(y), x)

    def sat(self):
        n = self.N
        adj = self.adj

        index = 0
        stack = []
        onstack = [False] * n
        ids = [-1] * n
        low = [0] * n
        comp = [-1] * n
        self.time = 0
        self.comp_id = 0

        def dfs(u):
            nonlocal index

            ids[u] = low[u] = index
            index += 1
            stack.append(u)
            onstack[u] = True

            for v in adj[u]:
                if ids[v] == -1:
                    dfs(v)
                    low[u] = min(low[u], low[v])
                elif onstack[v]:
                    low[u] = min(low[u], ids[v])

            if low[u] == ids[u]:
                while True:
                    v = stack.pop()
                    onstack[v] = False
                    comp[v] = self.comp_id
                    if v == u:
                        break
                self.comp_id += 1

        for i in range(n):
            if ids[i] == -1:
                dfs(i)

        for i in range(0, n, 2):
            if comp[i] == comp[i ^ 1]:
                return False

        return True
```

10.0.4. *Dpll.*

```
class SAT:
    def __init__(self):
        self.clauses = []
        self.n = 0

    def var(self):
        self.n += 1
        return self.n

    def clause(self, c):
        self.clauses.append(c)

    def clause_sat(self, clause, assign):
        unknown = False

        for x in clause:
            v = abs(x)
            val = assign[v]

            if x > 0 and val == 1:
                return True
            if x < 0 and val == 0:
                return True

            if val == -1:
                unknown = True

        return False if not unknown else None

    def propagate(self, assign):
        changed = True

        while changed:
            changed = False

            for clause in self.clauses:
                sat = False
                unassigned = []

                for x in clause:
                    v = abs(x)

                    if x > 0 and assign[v] == 1:
                        sat = True
                        break
                    if x < 0 and assign[v] == 0:
                        sat = True
                        break

                    if assign[v] == -1:
                        unassigned.append(x)

                if sat:
                    continue

                if len(unassigned) == 0:
                    return False  # contradiction

                if len(unassigned) == 1:
                    x = unassigned[0]
```

```
------------------- v = abs(x) -------------------------------------
------------------- val = 1 if x > 0 else 0 -------------------------
 ------------------------------------------------------------------
------------------- if assign[v] != -1 and assign[v] != val: ---------
----------------------- return False -----------------------------
 ------------------------------------------------------------------
------------------- if assign[v] == -1: ------------------------------
----------------------- assign[v] = val ------------------------------
----------------------- changed = True -------------------------------
 ------------------------------------------------------------------
------- return True -------------------------------------------------
 ------------------------------------------------------------------
--- def solve_dfs(self, assign, idx=1): ------------------------------
------- if idx > self.n: -------------------------------------------
----------- return True ------------------------------------------
 ------------------------------------------------------------------
------- if assign[idx] != -1: --------------------------------------
----------- return self.solve_dfs(assign, idx + 1) -------------------
 ------------------------------------------------------------------
------- for val in [0, 1]: -----------------------------------------
----------- new_assign = assign[:] -------------------------------
----------- new_assign[idx] = val --------------------------------
 ------------------------------------------------------------------
----------- if self.propagate(new_assign): -----------------------
--------------- if self.solve_dfs(new_assign, idx + 1): --------------
------------------- return True ------------------------------------
 ------------------------------------------------------------------
------- return False ------------------------------------------------
 ------------------------------------------------------------------
--- def solve(self): -------------------------------------------------
------- assign = [-1] * (self.n + 1) -------------------------------
 ------------------------------------------------------------------
------- if not self.propagate(assign): ------------------------------
----------- return False ------------------------------------------
 ------------------------------------------------------------------
------- return self.solve_dfs(assign) -------------------------------
 ------------------------------------------------------------------
--- def get_value(self, x): ------------------------------------------
------- return self.assign[x] == 1 ----------------------------------
```